

โครงการอบรมเชิงปฏิบัติการ Upskill/Reskill

เอกสารพัฒนาเฟิร์มแวร์ USB2CAN-FD

ภาควิชาวิศวกรรมไฟฟ้าและคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์

2026-04-19

1. Firmware Requirements

USB2CAN คือเฟิร์มแวร์สำหรับบอร์ด NUCLEO-C562RE ที่ทำหน้าที่เป็น ตัวแปลงสัญญาณ (Adapter) ระหว่างพอร์ต USB และเครือข่าย CAN FD โดยใช้โปรโตคอล **SLCAN (Serial Line CAN)** ซึ่งเป็นมาตรฐานสากลที่ช่วยให้อุปกรณ์สามารถเชื่อมต่อกับซอฟต์แวร์วิเคราะห์เครือข่าย เช่น CANgaroo, SavvyCAN ได้

1.1 Interface Specification

Pins	Function	Interface
PB8 / PB9	FDCAN1_RX / FDCAN1_TX	ขา TXD/RXD ของ MCP2562FD
PE2	STBY	ขา Standby (Active High) ของ MCP2562FD
PA2 / PA3	VCP_TX / VCP_RX	ST-Link สำหรับสื่อสารกับ PC ด้วยสถานะ 115200,8,n,1

1.2 SLCAN Commands

ชุดคำสั่งสำหรับการควบคุมและรับส่งข้อมูล CAN ผ่านพอร์ตซีเรียล มีรูปแบบดังนี้:

คำสั่ง (Command)	รูปแบบ (Syntax)	คำอธิบาย (Description)	ตัวอย่าง (Example)
Open Port	O↵	เปิดการทำงานของพอร์ต CAN	O↵
Close Port	C↵	ปิดการทำงานของพอร์ต CAN	C↵
Setup Btrrate	S{n}↵	ตั้งค่า Nominal Btrrate (มาตรฐาน 2.0)	S6↵ (500 kbps)
Data Rate (FD)	Y{n}↵	ตั้งค่า CAN-FD Data Btrrate (BRS)	Y8 (4 Mbps)
Transmit Standard	t{id}{len}{data}↵	ส่ง Data Frame (Standard ID)	t123411223344↵
Transmit Extended	T{id}{len}{data}↵	ส่ง Data Frame (Extended ID)	T0000012381122...↵

คำสั่ง (Command)	รูปแบบ (Syntax)	คำอธิบาย (Description)	ตัวอย่าง (Example)
Transmit FD	fb{id}{len} {data}↵	ส่ง CAN FD Frame (BRS Enabled)	fb1230C11...↵
Listen Mode	L↵	เปิดโหมด Monitor อย่างเดียว (ไม่ส่ง ACK)	L↵
Version Query	V↵	สอบถามเวอร์ชันของเฟิร์มแวร์	V↵

1.2.1 ลำดับการตั้งค่า

1. **Setup Bitrate (S หรือ Y):** ผู้ใช้ส่งคำสั่งตั้งความเร็ว (เช่น S6 สำหรับ 500kbps) ในขณะที่พอร์ตยังปิดอยู่
2. **Open Port (O):** เฟิร์มแวร์จะสั่งให้ FDCAN เริ่มทำงาน และเปลี่ยนสถานะชิป Transceiver จาก Standby เป็น Normal
3. **Acknowledge:** เฟิร์มแวร์ตอบกลับด้วยอักขระ \r (CR) หรือ \a (Bell) หากคำสั่งสำเร็จหรือล้มเหลว

1.2.2 ลำดับการรับ CAN message

1. **HW Trigger:** ข้อมูลเข้าสู่ RX FIFO ของ FDCAN
2. **FIFO Interrupt:** CPU คัดลอกข้อมูลจาก FIFO ลง Circular Buffer
3. **Parsing:** Main Loop แปลง Binary เป็นข้อความ ASCII ตามรูปแบบ SLCAN
4. **Transmission:** ส่งข้อความผ่าน UART (VCP) ไปยัง PC

1.2.3 ลำดับการส่ง CAN message

1. **Receive CMD:** เฟิร์มแวร์รับคำสั่งจาก UART (เช่น t1232AABB\r) จนพบตัวปิดคำสั่ง \r
2. **Validation:** Parser ตรวจสอบ ID ความยาวข้อมูล (DLC) สอดคล้องกับข้อมูลที่ส่งมาหรือไม่?
3. **Hardware Write:** แปลงข้อความ ASCII กลับเป็น Binary แล้วเขียนลงใน TX FIFO ของ FDCAN
4. **Bus Arbitration:** FDCAN รอจังหวะว่างของบัสเพื่อส่งออกไปตามโปรโตคอล CAN
5. **Confirmation:** เมื่อส่งสำเร็จ เฟิร์มแวร์จะส่ง \r กลับไปที่ PC เพื่อยืนยันว่าคำสั่งส่งออกไปแล้ว

1.3 Functional Requirements

1. **Dual-Rate Support:** ระบบต้องสามารถตั้งค่าแยกกันระหว่าง Nominal Bitrate สำหรับ arbitration และ Data Bitrate สำหรับการส่งผ่านข้อมูล
2. **Buffer Management:** เนื่องจากความเร็วของ CAN FD สูงกว่า UART มาก ระบบต้องมี **Hardware FIFO** และ **Software Circular Buffer** ขนาดใหญ่อย่างน้อย 2KB เพื่อป้องกันข้อมูลสูญหาย (Data Drop)
3. **Timestamp Accuracy:** ในโหมดรับข้อมูล (Monitor) เฟิร์มแวร์ต้องระบุ Timestamp ระดับ Microseconds (us) แนบไปกับข้อมูลแต่ละเฟรมเพื่อใช้ในการวิเคราะห์ Latency
4. **Visual Indicator (LED / PA5):** กระพริบสม่ำเสมอเมื่อระบบทำงานปกติ

2. Firmware Design

การออกแบบเฟิร์มแวร์ USB2CAN มุ่งเน้นไปที่การสื่อสารแบบ Real-time ระหว่างบัส CAN FD และพอร์ต UART โดยใช้โปรโตคอล SLCAN เป็นสื่อกลาง เพื่อให้สามารถทำงานร่วมกับซอฟต์แวร์วิเคราะห์มาตรฐานได้

2.1 Software Architecture

การทำงานของเฟิร์มแวร์แบ่งออกเป็น 3 เลเยอร์หลัก เพื่อแยกส่วนการติดต่อฮาร์ดแวร์ออกจากลอจิกของโปรโตคอล:

1. **Application Layer:** จัดการ State Machine หลักของระบบและการจัดการ Error
2. **Service Layer:** ประกอบด้วย SLCAN Parser (ตีความคำสั่ง) และ Circular Buffer (จัดการคิวข้อมูล)
3. **Hardware Abstraction Layer (HAL):** จัดการ FDCAN (Peripheral) และ UART (VCP)

2.2 Behavior Design: Main State Machine

การทำงานหลักถูกควบคุมด้วย State Machine เพื่อควบคุมการสื่อสารกับองค์ประกอบ FDCAN:

เหตุการณ์ / สถานะ	IDLE	NORMAL	MONITOR	ERROR
OPEN	NORMAL / init FDCAN	–	–	–
CLOSE	–	IDLE / reset FDCAN	–	–
RATE	IDLE / config FDCAN			
LISTEN	–	MONITOR / config FDCAN	NORMAL / config FDCAN	–
TX_MSG	–	NORMAL / send MSG	–	–
RX_MSG	–	NORMAL / report MSG	NORMAL / report MSG	–
error	–	ERROR / reset FDCAN	ERROR / reset FDCAN	

2.3 SLCAN Command: Parser State Machine

SLCAN เป็นโปรโตคอลแบบ ASCII-based ที่ลงท้ายด้วย \r (<-) จึงทำงานแบบ Non-blocking เพื่อไม่ให้ขัดจังหวะการรับข้อมูลจากบัส CAN:

State	Description	Transition Trigger
WAIT_CMD	รอรับตัวอักษรแรก (เช่น t, T, O, S)	เมื่อพบอักษรตัวแรกที่ต้องการ
COLLECT_DATA	เก็บข้อมูล ID, DLC และ Data เข้าสู่ Buffer	เมื่อพบอักษรตัวถัดไปจนครบกำหนด
VALIDATE	ตรวจสอบความถูกต้องของความยาวข้อมูล (Checksum/Length)	เมื่อตรวจสอบเสร็จสิ้น
DISPATCH	ส่งข้อมูลเข้า FDCAN TX Buffer หรือ เปลี่ยนค่า Config	เมื่อประมวลผลสำเร็จ

2.4 Timing analysis

การประเมินว่าเฟิร์มแวร์สามารถรองรับ CAN FD ที่มีความเร็วสูง (สูงสุด 8 Mbps) ได้โดยไม่เกิด Data Loss:

1. **UART Bottleneck:** ที่ 115,200 bps สามารถส่งข้อมูลได้ประมาณ 11.5 KB/s หาก CAN บัสมี Traffic หนาแน่น ต้องพิจารณาเพิ่ม Baud rate เป็น **921,600 bps** หรือใช้ **USB CDC High Speed**
2. **Interrupt Latency:** การจัดการข้อมูลใน FDCAN_IT_RX_FIFO0 ต้องทำให้น้อยที่สุด โดยการย้ายข้อมูลลง Circular Buffer ทันทีและปล่อยให้การจัดรูปแบบข้อความ ASCII เป็นหน้าที่ของ Main Loop (Background Task)

3. **Cyclic Task:** Main Loop จะทำงานที่ความถี่สูงกว่า 1ms (1kHz) เพื่อตรวจสอบคำสั่งจาก UART และระบายข้อมูลจาก CAN Buffer ออกสู่ PC ให้เร็วที่สุด

3. Firmware Detailed Design

โครงสร้างเฟิร์มแวร์ USB2CAN ถูกออกแบบให้มีความยืดหยุ่นสูง (Scalability) เพื่อรองรับการสลับระหว่างโหมด CAN 2.0 และ CAN FD โดยเน้นการจัดการข้อมูลผ่าน Buffer เพื่อป้องกันปัญหาคอขวดระหว่าง Bus ความเร็วสูงและ Serial Interface

3.1 Firmware Architecture

การจัดโครงสร้างไฟล์ถูกแยกส่วนตามบทบาท (Layered Architecture) เพื่อให้ง่ายต่อการบำรุงรักษาและทดสอบ:

Source files	Responsibility	Key Functions
main.c	จัดการ Main Loop และควบคุมระบบ SLCAN State Machine	main(), System_Init()
slcan_parser.c/h	ตีความคำสั่ง ASCII (SLCAN) และแปลงเป็น CAN Frame	SLCAN_Parse(), SLCAN_Transmit()
can_service.c/h	ควบคุม FDCAN Peripheral, Filter และการตั้งค่า Bittare	CAN_SetBittare(), CAN_Open(), CAN_Close()
ring_buffer.c/h	จัดการคิวข้อมูล (Queue) ระหว่าง Interrupt และ Main Loop	RingBuf_Put(), RingBuf_Get()
stm32c5xx_it.c	จัดการ Interrupt Service Routines สำหรับ UART และ FDCAN	FDCAN1_IT0_IRQHandler(), USART2_IRQHandler()

3.2 Main Loop Execution Sequence

ลำดับการทำงานในลูปหลักจะเน้นการตรวจสอบสถานะ (Polling) และระบายข้อมูลออกจาก Buffer อย่างรวดเร็ว:

1. **Check UART RX:** ตรวจสอบว่ามีคำสั่ง SLCAN (เช่น t, 0, C) เข้ามาหรือไม่
2. **Execute Command:** หากได้รับคำสั่งที่สมบูรณ์ (\r \n) Parser แปลงเป็น event
3. **State machine:** ประมวลผล state machine
4. **Check CAN RX Buffer:** ตรวจสอบว่ามีข้อมูล CAN ที่ถูกพักไว้ใน Ring Buffer หรือไม่
5. **Format & Send:** หากมีข้อมูล ให้แปลงจาก Binary เป็นสตริง SLCAN (เช่น t123...) และส่งออกทาง UART
6. **Heartbeat & Status:** อัปเดตสถานะไฟ LED (PA5) เพื่อแสดงความพร้อมของระบบ

3.3 HW Configuration Summary

การตั้งค่าเพอร์ิเฟอรัลด้วย STM32CubeMX2 สำหรับ USB2CAN:

องค์ประกอบ (Peripheral)	โหมด (Mode)	การตั้งค่า (Key Settings)	วัตถุประสงค์ (Purpose)
FDCAN1	Normal Mode	Nominal=500kbps, Data=2Mbps	รับ-ส่งข้อมูลบนบัส CAN FD
FDCAN Filter	Range/Mask	Filter 0: Accept All	รับข้อมูลทุก Frame ในช่วงแรก

องค์ประกอบ (Peripheral)	โหมด (Mode)	การตั้งค่า (Key Settings)	วัตถุประสงค์ (Purpose)
TIM2	32-bit Counter	Frequency = 1 MHz	สร้าง Timestamp หน่วย Microsecond
USART2	Asynchronous	115200 8N1 (Interrupt Enabled)	สื่อสาร SLCAN กับเครื่องคอมพิวเตอร์
GPIO PE2	Output	High (Standby), Low (Normal)	ควบคุมโหมดของ MCP2562FD

3.4 Implementation Details

3.4.1 SLCAN Data Structure

การประกาศโครงสร้างข้อมูลสำหรับจัดเก็บ CAN Frame เพื่อใช้ในการ Parsing:

```
/* SLCAN Frame Representation */
typedef struct {
    uint32_t id;           // Identifier (Standard/Extended)
    uint8_t  len;          // Data Length Code (DLC)
    uint8_t  data[64];     // Payload ( 64 byte CAN FD)
    bool     is_fd;        // Flag CAN FD
    bool     is_brs;       // Flag (Bit Rate Switch)
} SLCAN_Frame_t;

typedef struct {
    UART_HandleTypeDef *huart;    // UART handler
    FDCAN_HandleTypeDef *hfdcan;  // FDCAN handler
    GPIO_TypeDef *stby_port;      // GPIO handler for standby
    uint16_t stby_pin;
    SystemState_t state;          // CONFIG/OPERATIONAL
} SLCAN_Handler_t;
```

3.4.2 SLCAN command-API mapping

รายการของฟังก์ชันที่เชื่อมโยงจากคำสั่ง

```
/* 1. Lifecycle Management */
int slcan_init(SLCAN_Handler_t *hslcan, UART_HandleTypeDef *huart, FDCAN_HandleTypeDef *hfdcan);
int slcan_deinit(SLCAN_Handler_t *hslcan);

/* 2. Connection & Channel Control */
int slcan_open(SLCAN_Handler_t *hslcan);
int slcan_close(SLCAN_Handler_t *hslcan);
int slcan_is_open(SLCAN_Handler_t *hslcan);

/* 3. Configuration & Bitrate */
int slcan_setup_bitrate(SLCAN_Handler_t *hslcan, uint8_t index);
int slcan_setup_bitrate_fd(SLCAN_Handler_t *hslcan, uint8_t index);
int slcan_set_listen_only(SLCAN_Handler_t *hslcan, bool on);
```

```
/* 4. Data Transmission (Non-blocking) */
//      :   Pointer   SLCAN_Frame_t   slcan_message_t
int slcan_transmit(SLCAN_Handler_t *hslcan, const SLCAN_Frame_t *frame);
int slcan_process_rx(SLCAN_Handler_t *hslcan);

/* 5. Filtering (Acceptance Control) */
int slcan_set_acceptance_code(SLCAN_Handler_t *hslcan, uint32_t code);
int slcan_set_acceptance_mask(SLCAN_Handler_t *hslcan, uint32_t mask);

/* 6. Diagnostic & Information */
int slcan_get_status(SLCAN_Handler_t *hslcan, uint32_t *flags);
int slcan_get_version(SLCAN_Handler_t *hslcan, uint8_t *hw, uint8_t *sw);
int slcan_get_serial(SLCAN_Handler_t *hslcan, uint32_t *sn);
```

4. Firmware Testing

การทดสอบเฟิร์มแวร์ USB2CAN มุ่งเน้นไปที่การตรวจสอบความถูกต้องของโปรโตคอล SLCAN และความสามารถในการสื่อสารกับอุปกรณ์มาตรฐาน (Standard Interoperability) ผ่านเครือข่าย CAN FD

4.1 Test Environment

การจัดเตรียมสภาพแวดล้อมเพื่อทดสอบความเสถียรและการรับส่งข้อมูล (Test Setup):

- 1. **Hardware Setup:**
 - **Target Board:** บอร์ด NUCLEO-C562RE ที่ติดตั้งเฟิร์มแวร์ USB2CAN
 - **Reference Device:** บอร์ด WeAct USB2CANFD (ใช้เป็นตัวแทนเปรียบเทียบและเครื่องกำเนิดสัญญาณ CAN)
 - **Bus Connection:** เชื่อมต่อขา CAN_H และ CAN_L ของทั้งสองบอร์ดเข้าด้วยกัน (ตรวจสอบการติดตั้ง 120Ω Terminator ที่ปลายบัส)
- 2. **Software Setup:**
 - **Reference Software:** CANGaroo (สำหรับควบคุม WeAct USB2CANFD)
 - **Test Terminal:** Serial Monitor (เช่น Tera Term หรือ PuTTY) สำหรับส่งคำสั่ง SLCAN ด้วยตนเอง
 - **Protocol Analyzer:** ใช้การวิเคราะห์สตรีม ASCII ที่ได้รับเพื่อตรวจสอบรูปแบบ SLCAN

4.2 Unit Testing

ตารางสรุปการทดสอบฟังก์ชันพื้นฐานของโปรโตคอล SLCAN:

ID	Objective	Pre-req	Steps	Input	Expected Output
UT-01	Test Port Opening	UART Connected	ส่งคำสั่งเปิดพอร์ต	0↵	\r (CR) และ LED PE2 เป็น LOW
UT-02	Bitrate Config	Port Closed	ตั้งความเร็ว 500 kbps	S6↵	\r (CR)

ID	Objective	Pre-req	Steps	Input	Expected Output
UT-03	CAN Transmit	Port Opened	ส่ง Standard Frame ID 0x123	t1232AABB↔	\r และข้อความปรากฏบน CANGaroo
UT-04	Invalid Cmd	Any State	ส่งคำสั่งที่ไม่มีในสารบบ	X↔	\a (Bell Character)
UT-05	CAN FD Config	Port Closed	ตั้ง Data Rate 2 Mbps	Y7↔	\r (CR)

4.3 Integration Testing (System Validation)

การทดสอบความร่วมมือระหว่างเฟิร์มแวร์และแอปพลิเคชันภายนอกภายใต้สถานการณ์จริง:

4.3.1 Loopback Testing (Inter-device)

- **Purpose:** ตรวจสอบความถูกต้องของการแปลงข้อมูลจาก UART เป็น CAN และกลับมาเป็น UART
- **Procedure:** 1. ใช้ CANGaroo สั่งให้ WeAct ส่งข้อมูล ID 0x7FF ข้อมูล 0x11 0x22
2. สังเกตข้อมูลที่ปรากฏใน Serial Monitor ของ Nucleo
- **Expected Result:** ต้องได้รับสตริง t7FF21122\r พร้อม Timestamp ที่มีความละเอียดระดับ Microsecond

4.3.2 Stress Test & Buffer Stability

- **Purpose:** ตรวจสอบความทนทานเมื่อมี Traffic บนบัสหนาแน่น (Bus Load > 50%)
- **Procedure:** 1. ใช้ CANGaroo ตั้ง "Periodic Send" ทุกๆ 1ms
2. ตรวจสอบว่าเฟิร์มแวร์ Nucleo ยังสามารถส่งข้อมูลออกจาก UART ได้ต่อเนื่องหรือไม่
- **Expected Result:** ข้อมูลใน Serial Monitor ต้องไม่ขาดหาย และไม่เกิดอาการเครื่องค้าง (Internal Error)

4.3.3 CAN FD BRS (Bit Rate Switch) Validation

- **Purpose:** ตรวจสอบความสามารถในการสลับความเร็วในสถานะ CAN FD
- **Procedure:** 1. ตั้งค่าทั้งสองฝั่งให้รองรับ CAN FD (Nominal 500k, Data 2M)
2. ส่งเฟรมด้วยคำสั่ง fb
- **Expected Result:** บอร์ด WeAct (ผ่าน CANGaroo) ต้องสามารถรับและถอดรหัสข้อมูลความเร็วสูงได้ถูกต้องโดยไม่มี Error Frame